

Monte Carlo Tree Search

Lecture 10

CS 4880

Kirk Duran

Beyond Deep Blue: When Full-Width Search Breaks

Minimax and alpha–beta pruning work well when the game tree is manageable and a strong evaluation function is available.

- ▶ Chess has a large tree, but tactical structure and move ordering make pruning effective.
- ▶ Some games have much larger branching factors or unreliable handcrafted evaluation functions.
- ▶ Randomness or hidden information can further multiply the number of outcomes that must be considered.

A different strategy

Instead of expanding every relevant continuation, can we **sample possible futures** and concentrate computation where it matters?

Visual placeholder: chess tree transitioning into a much wider Go-like tree, with only a small sampled region highlighted.

Key idea

Monte Carlo search trades exhaustive enumeration for statistical estimates built from repeated simulations.

Why Go Challenged Classical Game Search

Go is difficult for full-width adversarial search because its tree is both **wide** and **deep**.

- ▶ A typical early position may offer hundreds of legal moves.
- ▶ The number of legal board configurations is on the order of 10^{170} .
- ▶ Local moves can have global strategic consequences many turns later.
- ▶ Exhaustive search to terminal states is completely infeasible.

Consequence

Even dramatic pruning leaves far too many continuations to explore uniformly.

Visual placeholder: chess board beside Go board, with a narrow chess tree and dramatically wider Go tree.

Why Alpha-Beta Is Not Enough

Alpha-beta pruning is exact, but its efficiency depends heavily on the structure of the search.

- ▶ Good move ordering is required for aggressive pruning.
- ▶ A depth limit still requires an informative evaluation function.
- ▶ In Go, many positions are strategically ambiguous until much later.
- ▶ Searching every move to the same depth wastes effort on weak or irrelevant branches.

Shift in viewpoint

Rather than ask “How deep can we search every move?”, ask “Which moves deserve more samples?”

Visual placeholder: a full-width alpha-beta tree contrasted with an uneven tree that allocates more depth to promising branches.

Key idea

MCTS is a selective search method: computation is allocated adaptively rather than uniformly by depth.

Reminder: Deterministic vs. Stochastic Games

Deterministic adversarial games

- ▶ Actions have deterministic outcomes.
- ▶ Minimax models an opponent choosing the worst response for MAX.
- ▶ Example: chess under perfect-information assumptions.

Stochastic games

- ▶ Some transitions are random.
- ▶ Chance nodes represent probability distributions over outcomes.
- ▶ Examples: dice, shuffled cards, random events.

Visual placeholder: chess piece on the left; dice/cards on the right; arrows showing deterministic choice versus probabilistic outcome.

Expectiminimax Recap

Definition

Expectiminimax extends minimax with **chance nodes** whose values are probability-weighted expectations.

- ▶ MAX node: choose the child with maximum value.
- ▶ MIN node: choose the child with minimum value.
- ▶ CHANCE node: compute

$$V(s) = \sum_i P(s_i \mid s) V(s_i).$$

Assumption

The probability model for each chance outcome is known and can be enumerated.

Example placeholder: MAX node above CHANCE nodes, then MIN nodes and terminal utilities. Label chance edges with probabilities.

Expectiminimax: One Worked Backup

Suppose a chance event selects between two MIN states with equal probability.

$$\min(3, 5) = 3, \quad \min(2, 4) = 2.$$

Therefore the chance node has value

$$0.5(3) + 0.5(2) = 2.5.$$

- ▶ Exact expectation requires both outcomes.
- ▶ More outcomes mean more branches to enumerate.

Example placeholder: chance node with two 0.5 branches to MIN nodes valued 3 and 2, backing up expected value 2.5.

Key idea

Expectation is easy to compute when the outcome set is small; it becomes expensive when uncertainty creates many possible futures.

Stochastic Games Multiply the Branching Factor

Random events can make a game tree much wider than the action set alone suggests.

- ▶ Backgammon combines legal moves with dice outcomes.
- ▶ Card games combine actions with hidden deals and later draws.
- ▶ Each additional chance outcome creates another continuation to evaluate.

Effective branching

A player branch followed by a chance branch behaves roughly like the product of their branching factors.

Visual placeholder: backgammon board with a decision node followed by a large fan-out of dice outcomes.

The Limitation of Exhaustive Expectation

Exact expectiminimax inherits the exponential growth of minimax and adds chance branching.

- ▶ Every modeled outcome must be expanded.
- ▶ Low-probability outcomes still consume search effort.
- ▶ Exact probabilities may be expensive or unknown.
- ▶ A fixed-depth tree does not naturally provide a useful answer if interrupted early.

Question

Can we estimate an expectation from a **representative sample** instead of enumerating every outcome?

Visual placeholder: exhaustive expectimax tree with many faint chance branches; only several sampled trajectories emphasized.

From Enumeration to Sampling

Consider again a chance node with two equally likely outcomes worth 3 and 2.

- ▶ Exact value: $0.5(3) + 0.5(2) = 2.5$.
- ▶ Sample one outcome: estimate is either 3 or 2.
- ▶ Sample repeatedly and average: the estimate approaches 2.5.

Monte Carlo principle

Approximate an expectation using empirical averages from random samples.

Example placeholder: same expectiminimax tree, but show repeated sampled paths and a running average converging toward 2.5.

Key idea

Sampling replaces complete knowledge of the tree with progressively better statistical estimates.

Monte Carlo Tree Search: High-Level Idea

Definition

Monte Carlo Tree Search (MCTS) incrementally builds a partial search tree by repeatedly selecting, expanding, simulating, and backing up sampled outcomes.

- ▶ Promising actions receive more simulations.
- ▶ Poorly performing actions gradually receive less attention.
- ▶ The tree becomes deeper where evidence suggests depth is valuable.
- ▶ Search can be stopped at almost any time and still return a current best move.

Modern variants

MCTS can use random rollouts, heuristic rollouts, or learned policy/value models.

Visual placeholder: asymmetric MCTS tree with a few heavily visited branches and many lightly explored alternatives.

The Multi-Armed Bandit Problem

Imagine k slot-machine arms with unknown reward distributions.

- ▶ Pulling an arm reveals a reward sample.
- ▶ We want high total reward over repeated pulls.
- ▶ The best-known arm may not truly be the best arm.

The central tension

Exploit actions that currently look strong, or **explore** actions whose value is still uncertain.

Visual placeholder: three slot-machine arms labeled A, B, C with unknown reward distributions and a question mark over which to choose.

Key idea

Every interior MCTS node creates a local bandit problem over its available actions.

Bandit Example: Which Arm Next?

Suppose we have observed:

Arm	Visits	Total reward	Mean
A	3	2	0.67
B	2	1	0.50
C	1	1	1.00

- ▶ C has the highest observed mean.
- ▶ C is also the least tested.
- ▶ A has more evidence, but a lower mean.

Example placeholder: three arms annotated with visit counts and rewards; ask "Which arm should be pulled next?"

Upper Confidence Bounds

Definition

UCB1 scores each action using its estimated reward plus an uncertainty bonus.

$$\text{UCB1}(i) = \underbrace{\frac{Q_i}{N_i}}_{\text{exploitation}} + C \underbrace{\sqrt{\frac{\ln N}{N_i}}}_{\text{exploration}}.$$

- ▶ Q_i : cumulative reward observed for action i .
- ▶ N_i : number of times action i has been selected.
- ▶ N : total selections at the parent.
- ▶ $C > 0$: exploration constant controlling the tradeoff.

Key idea

Frequently successful actions score well because of their mean reward; rarely tested actions score well because uncertainty remains high.

UCB1 Intuition: Revisit or Explore?

Exploitation

Return to an option that has already produced strong average rewards.

- ▶ evidence of quality;
- ▶ lower uncertainty;
- ▶ immediate payoff.

Exploration

Try an option that has received fewer samples.

- ▶ information gain;
- ▶ higher uncertainty;
- ▶ possibility of discovering a better action.

Analogy placeholder: familiar food stall with a proven rating versus a new stall with little evidence but uncertain potential.

UCB1 Worked Example

Using the bandit statistics from the previous slide, let $N = 6$ and $C = \sqrt{2}$.

$$\text{UCB1}(A) = \frac{2}{3} + \sqrt{2} \sqrt{\frac{\ln 6}{3}} \approx 1.76,$$

$$\text{UCB1}(B) = \frac{1}{2} + \sqrt{2} \sqrt{\frac{\ln 6}{2}} \approx 1.84,$$

$$\text{UCB1}(C) = 1 + \sqrt{2} \sqrt{\ln 6} \approx 2.89.$$

Decision

Choose C. Its observed reward is strong, and its single visit gives it the largest uncertainty bonus.

Key idea

UCB does not explore randomly; it explores where uncertainty is still plausibly valuable.

From UCB1 to UCT

MCTS applies the bandit idea recursively inside a search tree.

- ▶ Each state is a node.
- ▶ Each legal action is an arm.
- ▶ Simulated outcomes provide reward samples.
- ▶ The UCB-style rule used in trees is commonly called **UCT**: Upper Confidence bounds applied to Trees.

Adversarial detail

Values must use a consistent player perspective. At opponent nodes, rewards are typically negated or stored from the player-to-move perspective.

Visual placeholder: one search-tree node transformed into a local multi-armed-bandit decision over its child actions.

The Four Phases of MCTS

1. **Selection:** follow the tree from the root using UCT until reaching a node that is not fully expanded.
2. **Expansion:** add one or more previously unvisited actions as new child nodes.
3. **Simulation / Evaluation:** estimate the new node's outcome using a rollout or value model.
4. **Backpropagation:** update reward statistics and visit counts along the selected path.

Repeat

Each iteration contributes one additional sample of evidence to the tree.

Diagram placeholder: tree path labeled 1 Selection, new child labeled 2 Expansion, rollout trajectory labeled 3 Simulation, upward arrows labeled 4 Backpropagation.

MCTS Pseudocode

Algorithmic structure

```
while budget remains:
    node ← root
    // SELECTION
    while node is fully expanded and nonterminal:
        node ← child with best UCT score
    // EXPANSION
    if node is nonterminal:
        node ← EXPAND(node)
    // SIMULATION / EVALUATION
    reward ← ESTIMATE(node)
    // BACKPROPAGATION
    while node ≠ null:
        node.visits ← node.visits + 1
        node.value ← node.value + reward
        reward ← TRANSFORM_FOR_PARENT(reward)
        node ← node.parent
```

Key idea

The search tree is not generated in advance; it emerges from repeated evidence-gathering iterations.

MCTS from Scratch: Iteration 1

Begin with an unexpanded root s_0 .

1. Expand left child s_{10} .
2. Simulate from s_{10} .
3. Suppose the rollout is a win for the root player: reward 1.0.
4. Backpropagate the result.

$$N(s_{10}) = 1, Q(s_{10}) = 1.0$$

$$N(s_0) = 1, Q(s_0) = 1.0.$$

Trace convention

To isolate the statistics, this toy trace stores rewards from the root player's perspective.

Example placeholder: root s_0 with one child s_{10} ; annotate both nodes with $N = 1, Q = 1.0$.

MCTS from Scratch: Iteration 2

The right child has never been visited.

- ▶ Treat an unvisited action as having effectively infinite exploration priority.
- ▶ Expand s_{11} .
- ▶ Suppose its rollout is a draw: reward 0.5.

After backpropagation:

$$N(s_{10}) = 1, Q(s_{10}) = 1.0$$

$$N(s_{11}) = 1, Q(s_{11}) = 0.5$$

$$N(s_0) = 2, Q(s_0) = 1.5.$$

Example placeholder: root with two children, left labeled $N = 1, Q = 1.0$, right labeled $N = 1, Q = 0.5$, root $N = 2, Q = 1.5$.

MCTS from Scratch: Iteration 3

With $C = \sqrt{2}$ and $N(s_0) = 2$:

$$\text{UCT}(s_{10}) = 1.0 + \sqrt{2}\sqrt{\ln 2} \approx 2.18,$$

$$\text{UCT}(s_{11}) = 0.5 + \sqrt{2}\sqrt{\ln 2} \approx 1.68.$$

Select s_{10} and expand child s_{20} . Suppose the rollout is a loss: reward 0.0.

Node	N	Q
s_{20}	1	0.0
s_{10}	2	1.0
s_{11}	1	0.5
s_0	3	1.5

Key idea

A disappointing rollout lowers the empirical value of the branch without permanently eliminating it.

MCTS from Scratch: Iteration 4

At the root, both actions now have mean reward 0.5, but the less-visited action receives a larger exploration bonus.

$$\begin{aligned}\text{UCT}(s_{10}) &= \frac{1.0}{2} + \sqrt{2} \sqrt{\frac{\ln 3}{2}} \approx 1.55, \\ \text{UCT}(s_{11}) &= \frac{0.5}{1} + \sqrt{2} \sqrt{\ln 3} \approx 1.98.\end{aligned}$$

Select s_{11} , expand s_{22} , and suppose the rollout wins with reward 1.0.

$$N(s_{22}) = 1, Q(s_{22}) = 1.0; \quad N(s_{11}) = 2, Q(s_{11}) = 1.5; \quad N(s_0) = 4, Q(s_0) = 2.5.$$

What changed?

UCT redirected computation toward the less-certain branch, then updated its estimate using the new evidence.

What a Mature MCTS Tree Looks Like

After many iterations, the search tree becomes highly asymmetric.

- ▶ Strong actions receive many visits.
- ▶ Weak actions may remain shallow.
- ▶ Deep search emerges only in promising regions.
- ▶ The best current move can be returned when the time budget expires.

Typical final decision

Choose the root action with the largest visit count or strongest robust value estimate.

Visual placeholder: mature asymmetric tree, with edge thickness proportional to visits and only a few deep branches.

Key idea

MCTS spends computation according to evidence, not according to a uniform depth boundary.

The Risk of Random Rollouts

Uniformly random simulation is simple, but its estimates can be poor.

- ▶ **Variance:** a small number of lucky rollouts may distort the mean.
- ▶ **Bias:** random play may not resemble competent play.
- ▶ **Blind spots:** rare tactical sequences may almost never be sampled.
- ▶ **Waste:** obviously bad actions can consume rollout time.

Statistical consequence

Noisy rollouts require more samples before value estimates become reliable.

Visual placeholder: several rollout paths with widely varying outcomes; label variance, tactical blind spot, and wasted simulation.

Improving the Simulation Policy

Heuristic rollouts

Bias simulations toward plausible actions using domain knowledge.

Policy models

Use a learned policy to sample stronger, more realistic continuations.

Leaf evaluation

Stop the rollout early and estimate the state with a heuristic or learned value model.

- ▶ Better simulations can reduce variance and improve sample efficiency.
- ▶ Stronger guidance introduces model bias, so search and evaluation must be balanced carefully.

Key idea

Modern MCTS often replaces long random rollouts with learned policies and value estimates.

MCTS vs. Alpha-Beta Search

	Alpha-Beta	MCTS
Search shape	depth-oriented, full-width before pruning	asymmetric, sample-driven
Value estimate	heuristic evaluation at cutoff	empirical returns and/or learned value
Strength	deterministic games with strong evaluation and move ordering	enormous branching factors and any-time search
Weakness	sensitive to branching factor and evaluation quality	noisy estimates; may need many simulations
Decision budget	usually fixed depth / iterative deepening	naturally simulation- or time-bounded

Key idea

Neither method dominates universally: the best search algorithm depends on game structure, available models, and computational budget.

Games with Partial Observability

In imperfect-information games, the true state is not fully observed.

- ▶ Poker: opponents' private cards are hidden.
- ▶ Stratego: opponent piece identities are initially hidden.
- ▶ Real-time strategy games may contain fog of war.

Definition

A **belief state** represents uncertainty over possible underlying states consistent with the observations so far.

*Visual placeholder: observed public state
leading to several possible hidden worlds
inside a belief-state cloud.*

Key idea

Search must now reason over information, not only over physical game states.

Belief Sampling and Determinization

A simple way to adapt MCTS is to sample possible hidden states.

1. Sample a complete world consistent with current observations.
2. Run search as though that sampled world were the true state.
3. Repeat across many sampled worlds.
4. Aggregate the evidence for available actions.

Important caveat

Naive determinization can produce *strategy fusion*: it may choose different actions in sampled worlds that the real agent cannot distinguish.

Visual placeholder: one observed card-game state branching into several sampled hidden hands, each feeding a search tree.

MCTS in Poker-Like Games

Poker combines several complications at once:

- ▶ private information;
- ▶ stochastic card deals;
- ▶ adversarial betting decisions;
- ▶ opponent modeling and bluffing.

A practical MCTS-style system may:

1. sample hidden hands consistent with observations;
2. simulate betting and future cards;
3. back up utilities such as expected chip gain.

Example placeholder: poker table showing public cards, hidden opponent cards, sampled determinizations, and rollout branches for fold/call/raise.

Key idea

Imperfect-information search requires care: information-set methods are preferable to pretending hidden state is known.

Worked / Video Example: Poker Decision

Suggested in-class walkthrough:

- ▶ identify the information available to the acting player;
- ▶ list candidate actions: fold, call, raise;
- ▶ sample several plausible opponent holdings;
- ▶ trace one simulation for each candidate action;
- ▶ compare estimated expected utility after repeated samples.

Discussion question

What information would a naive determinization method accidentally give the agent that a real player does not possess?

Media placeholder: short poker hand clip or sequence of screenshots for an MCTS / belief-sampling walkthrough.

Why Belief-State Search Is Expensive

Many possible worlds

The number of hidden configurations may grow combinatorially with unknown cards, pieces, or states.

Belief updates

New observations require probabilities over possible worlds to be revised.

Planning over information

The agent must choose actions that work across states it cannot distinguish.

Opponent uncertainty

Good play may require reasoning about what the opponent knows and how the opponent acts.

Key idea

Partial observability adds a second search problem: reasoning about the state of knowledge itself.

AlphaGo: MCTS Guided by Neural Networks

AlphaGo demonstrated that MCTS becomes far stronger when search is guided by learned models.

- ▶ A **policy network** prioritizes promising moves during tree search.
- ▶ A **value network** estimates the probability of winning from a position.
- ▶ Search statistics refine these learned estimates through lookahead.
- ▶ The result is a focused tree that combines prior knowledge with online simulation.

Conceptual synthesis

Learning proposes and evaluates; MCTS allocates computation and tests consequences.

Visual placeholder: Go board feeding policy network and value network into an MCTS tree; optionally include AlphaGo documentary image.

Takeaways

- ▶ Full-width minimax and expectiminimax become impractical when branching factors are enormous.
- ▶ Monte Carlo methods estimate values from samples rather than enumerating every possible future.
- ▶ UCB1 and UCT balance **exploitation** of strong actions with **exploration** of uncertain actions.
- ▶ MCTS repeatedly performs **selection, expansion, simulation/evaluation, and backpropagation**.
- ▶ The resulting tree grows asymmetrically toward regions that appear promising.
- ▶ Rollout quality, value estimation, and information handling strongly affect performance.
- ▶ Neural guidance can make MCTS dramatically more sample-efficient, as demonstrated by AlphaGo.
- ▶ Next lecture: **Knowledge-Based Agents**.

Key idea

MCTS turns search into sequential evidence gathering: spend simulations where they are most likely to improve the decision.